

Binary Search — Pattern Recognition & Universal Notes

These notes are meant for quick revision before solving any binary search problem. They focus on recognizing when binary search applies, how to design the condition, and how to implement it safely.

1. How to Recognize a Binary Search Problem

Binary search is not only used on sorted arrays. The deeper idea is:

Binary search works when the answer space is ordered and a condition changes from false to true (or true to false).

Typical signals:

- "Find minimum / maximum such that condition holds"
- "Find smallest x satisfying..."
- "Find first / last occurrence"
- "Search in sorted array"
- "Answer lies between some low and high"
- "Constraints too large for linear scan"

Example patterns:

Problem	Binary Search On
Search element	index of sorted array
First/last occurrence	index
Minimum eating speed	answer space
Capacity to ship packages	answer space
Aggressive cows	distance
Median of two arrays	partition

If the solution space is ordered, binary search might work.

2. The Core Idea

Binary search repeatedly halves the search space.

Instead of checking every value:

We check the **middle value** and decide which half still contains the answer.

This works only when the property is **monotonic**.

Example monotonic behavior:

FFFFFFTTTT

or

TTTTTFFFFF

Binary search finds the boundary where it flips.

3. Classic Binary Search Template (Search in Sorted Array)

```
int binarySearch(vector<int>& arr, int target) {
    int l = 0;
    int r = arr.size() - 1;

    while (l <= r) {
        int mid = l + (r - l) / 2;

        if (arr[mid] == target)
            return mid;

        if (arr[mid] < target)
            l = mid + 1;
        else
            r = mid - 1;
    }
}
```

```
    return -1;
}
```

Time complexity:

$O(\log n)$

4. First True / Last False Pattern

Many problems reduce to:

"Find the first index where condition becomes true"

Template:

```
int l = 0, r = n - 1;
int ans = -1;

while (l <= r) {
    int mid = l + (r - l) / 2;

    if (condition(mid)) {
        ans = mid;
        r = mid - 1;
    }
    else {
        l = mid + 1;
    }
}

return ans;
```

This finds the **leftmost valid answer**.

5. Binary Search on Answer

Sometimes we cannot search indices.

Instead we search the **answer itself**.

Steps:

1. Define search space (low $\rightarrow$ high)
2. Write a function `can(x)`
3. Check if answer x is possible
4. Use binary search to find boundary

Example structure:

```
bool can(int x) {  
    // check if x works  
}  
  
int l = low, r = high;  
  
while (l <= r) {  
    int mid = l + (r - l) / 2;  
  
    if (can(mid))  
        r = mid - 1;  
    else  
        l = mid + 1;  
}  
  
return l;
```

Classic problems:

- Koko Eating Bananas
- Ship Packages Within D Days
- Minimum Days to Make Bouquets
- Aggressive Cows

6. Choosing the Search Space

This is the **most important design step**.

Examples:

Problem	Low	High
Eating speed	1	max pile
Ship capacity	max weight	sum of weights

Problem	Low	High
Aggressive cows	1	max distance

Always think:

"What is the smallest possible answer?"

"What is the largest possible answer?"

7. Avoiding Overflow

Never write:

```
mid = (l + r) / 2
```

Instead write:

```
mid = l + (r - l) / 2
```

This avoids integer overflow.

8. Two Main Binary Search Styles

Style 1

```
while (l <= r)
```

Used when searching for exact value.

Style 2

```
while (l < r)
```

Used when searching boundaries.

Example:

```
while (l < r) {  
    mid = (l + r) / 2  
  
    if (condition(mid))  
        r = mid  
    else  
        l = mid + 1  
}
```

9. Mental Checklist Before Coding

Before writing binary search ask:

1. Is the search space ordered?
2. Is there a monotonic condition?
3. What is the lowest possible answer?
4. What is the highest possible answer?
5. What does condition(mid) represent?

If these are clear, the problem becomes mechanical.

10. Pattern Recognition Summary

Binary search problems usually:

- Have a monotonic condition
- Ask for minimum/maximum valid value
- Work on sorted data or answer space
- Reduce search space by half each step

Think of binary search as:

finding the boundary where a property changes.

Once you see the boundary, the problem is solved.

11. Universal Binary Search Template

```
int binary_search_answer(int low, int high) {  
  
    while (low <= high) {  
        int mid = low + (high - low) / 2;  
  
        if (condition(mid))  
            high = mid - 1;  
        else  
            low = mid + 1;  
    }  
  
    return low;  
}
```

Use this template for most "minimum valid answer" problems.